IET Research Journals

Submission Template for IET Research Journal Papers

Adaptive PID Controller Based on Q-learning Algorithm

ISSN 1751-8644
doi: 0000000000
www.ietdl.org

Qian Shi[1*], *Hak-Keung Lam*[1*], *Bo Xiao*[2*], *Shun-Hung Tsai*[3]

[1] *Department of Informatics, King's College London, Bush House, Strand campus, 30 Aldwych, London, WC2B 4BG, United Kingdom*
[2] *Hamlyn Centre for Robotic Surgery, Imperial College London, London, SW7 2AZ, United Kingdom*
[3] *Graduate Institute of Automation Technology, National Taipei University of Technology, Taipei 10608, Taiwan*
* *E-mail: hak-keung.lam@kcl.ac.uk*
* *E-mail: qian.shi@kcl.ac.uk*
* *E-mail: b.xiao@imperial.ac.uk*

Abstract: An adaptive PID controller based on Q-learning algorithm is proposed in this paper to balance the cart-pole system in simulation environment. This controller was trained using Q-learning algorithm and implemented the learned Q-tables to change the gains of linear PID controllers according to the state of the system during the control process. The adaptive PID controller based on Q-learning algorithm was trained from a set of fixed initial positions and was able to balance the system starting from a series of initial positions that are different from the ones used in the training session, which achieved equivalent or even better performances in comparison with the conventional PID controller and the controller only uses Q-learning algorithm. This indicates the advantage of the adaptive PID controller based on Q-learning algorithm both in the generality of balancing the cart-pole system from a relatively wide range of initial positions and the stabilizability of achieving smaller steady-state error.

1 Introduction

Various controllers have been developed and widely used in industrial environments such as Proportional-Integral-Derivative (PID) controllers, fuzzy logic-based controllers because of the high robustness and scalability with the simplicity in structure. Nevertheless, these conventional controllers suffer from the high dependency on parameters. These controllers usually are not optimally tuned and able to reach satisfactory performances in real applications and require frequent adjustments when damages occur or being in adverse situations [1]. Therefore, tuning parameters of conventional controllers to achieve an optimal performance becomes an outstanding issue [2]. Additionally, these controllers have less satisfactory performance in complex environment, especially in the situations associated with practical applications considering the high nonlinearity and uncertainty of the environment. These drawbacks address the importance of developing alternative control techniques. Contrarily, Reinforcement Learning (RL) algorithm is a machine learning algorithm which generates optimal policy by interacting with environment and receiving reinforcement signals under the circumstance where the dynamics and the underlying of the environment remain unknown. This property proposes a promising solution to solving the systems with high nonlinearity by avoiding formulating accurate mathematical models [3]. Combing conventional control methods with RL is proved to be very powerful and efficient in solving practical problems like robotic control problems in high-dimensional state and action spaces [4]. There are several successful applications that have been implemented such as robot navigation [3], autonomous helicopter control [5] etc., showing a promising prospect of employing reinforcement learning algorithms in control systems.

Great amounts of researches have applied the RL algorithm to achieve innovative control strategies. One of them is to totally replace the conventional controllers by the RL algorithms. In [6], five different RL algorithms (two value-iteration algorithms and three policy-iteration algorithms) were tested to replace a double-loop PID controller to control a ball screw feed drive and all the algorithms indicated the superiority of the RL algorithms to the conventional PID controller. [7] used Q-learning algorithm to control the level of a liquid in a non-linear conical tank and a satisfactory performance was achieved. Though ideal achievements were observed in the applications stated above, it is also noticeable that since in RL algorithms the choose of state representation, output and reward all have intensive effects on the performance of the controller, designing controller by implementing RL algorithms individually becomes more challenging when the dimensions of the state and action space increase [2]. Another considerable approach is to combine the RL algorithm with the conventional controllers, where the RL algorithms are implemented to address the tuning issue that the conventional controllers have.

To solve this tuning issue, several systematic methods have been proposed in [8]. Ziegler–Nichols methods [9] proposed in 1995 is regarded as a classical tuning rule which has a broad implementation in industrial environment. Genetic algorithms were applied to provide an optimal conventional controller in the research described in [10]. Besides, a novel approach applied to optimize the parameters for nonlinear PID parameters called ant colony optimization algorithm were proposed and implemented in [11] and [12]. Both methods showed better performance compared with conventional controllers. However, most of the these approaches have high requirements on computational ability while RL algorithms significantly reduces the learning time [8].

Regarding to the RL based controllers, several techniques have been proposed in previous researches. A methodology named Discrete Action Reinforcement Learning Automata (DARLA) is proposed in 1997 in [13], two extension versions of this method were proposed in [14] and [15] aiming to tune the parameters of one PID controller with higher learning speed. Both controllers were tested on an Automatic Voltage Regulator (AVR) system. Similarly, Continuous Action Reinforcement Learning automata (CARLA) for PID controller [13] was applied as an on-line tuning method in the research done by [16] and [17]. In the research of [17], the approach was also tested via an engine idle-speed control system and the Ball and Beam System in simulation environment. Both showed the superior performance compared with the conventional controllers tuned following the Ziegler–Nichols methods. However, the integral and interpolation operation in the algorithm brings the extra cost of computation. Another method named actor-critic method used to tune

© The Institution of Engineering and Technology 2015

PID parameters was implemented in [18] as to implement wind turbine control and a similar method was implemented in [1] as to achieve tracking function in a special class of linear systems. In both algorithms, the Actor mapped states to PID controller parameters and the Critic provided the evaluation of the output of Actor. Both approaches indicated better performances contrasted to the conventional PID controllers. Meanwhile, Q-learning algorithm [19] is also widely used in tunning parameters for controllers. Researches [8] and [20] used this algorithm to tune fuzzy controllers. [2] tuned the PID controller using an incremental Q-PID algorithm by dynamically dividing the actions into more specific areas to obtain higher controlling accuracy. The simplicity in utilization and independency to the system parameters make Q-learning algorithm more reliable in real applications [14]. Nevertheless, less attention was paid to implementing Q-learning algorithm to tune parameters for multiple PID controllers. One related research is [21], which tuned two sets of PID controllers to control the angular and linear speed of a soccer robot and achieved good performance in speed response and stability. However, in this research the learning process of finding optimal parameters for PID controller required long period computation and less analysis were conducted on the learned controller regarding to the stability or robustness.

As to address the open issue of implementing Q-learning algorithm on multiple PID controllers. An adaptive PID controller based on Q-learning algorithm is proposed in this research. It adapts the similar approach implemented in [21] by varying the values of gains of linear PID controllers according to different operating space of system state after training with Q-learning algorithm, instead of having a set of fixed gains through the whole controlling progress. Generally, this innovative controller implements direct control through the PID controllers in the lower level while achieves adaptive adjustment of controller gains by learning through Q-learning algorithm in higher level, which improves the performance of the controllers in nonlinear and complex systems. Additionally, in order to have a more comprehensive analysis of the approach, the proposed controller was tested on the classical cart-pole problem in simulation environment in comparison with conventional PID controllers and the controller only using Q-learning algorithm. The main contributions of this paper are summarized as follows:

(1)An innovative structure of controller is proposed. The controller is consisted of two linear PID controllers with Q-learning algorithm applied to adaptively change the values of gains during the controlling process.
(2)The performance of the controller is tested on the cart-pole system in simulation environment. Both of the learning efficiency and the states of the cart-pole system are analyzed.
(3)Two other categories of controllers are provided as comparison, which are traditional linear PID controller and the controller only applies Q-learning algorithm. The comparison indicates the benefits of the adaptive PID controller in generality and stability.

The structure of rest of the paper is organized as follows. In Section 2, a brief introduction of relevant theories is conducted. The methodology and technical details are described in Section 3 and the simulation results of three different controllers are illustrated and compared in Section 4. Finally in Section 5, a conclusion is drawn and the future work is mentioned.

2 Preliminary

2.1 Linear PID Controller

Fig. 1 illustrates the structure of a typical linear PID controller, which consists of three components namely proportional, integral and derivative part. Equation (1) provides the output of the controller in discrete time form.

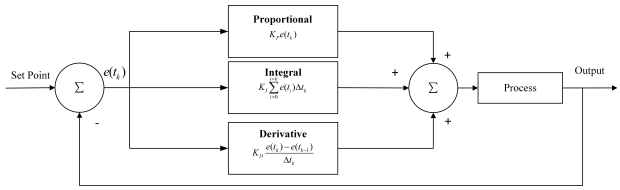

Fig. 1: Structure of PID controller

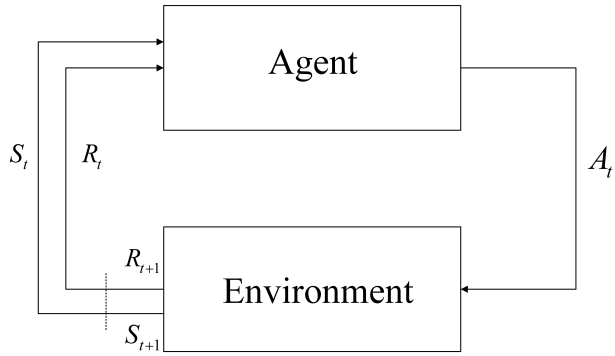

Fig. 2: Illustration of agent-environment interaction in RL

$$u(t_k) = K_P e(t_k) + K_I \sum_{i=0}^{k} e(t_i)\Delta t_k + K_D \frac{e(t_k) - e(t_{k-1})}{\Delta t_k} \tag{1}$$

where t_k is the kth time step; $u(t_k)$ is the output in k^{th} time step; $e(t_k)$ is the error in k^{th} time step; Δt_k is the interval of sampling time in simulation; K_P, K_I, K_D are the proportional, integral and derivative gain, respectively.

2.2 Reinforcement Learning

RL is a goal-directed problem solving approach which belongs to one of the approaches in machine learning. This approach was inspired by the nature of learning, which learns to generate control policy only by interacting with environment without knowing the underlying of the system model. The reinforcement learning problem can be defined as agent-environment interaction shown in Fig. 2. In this interaction, the agent observes states $S_t \in \mathcal{S}$ and takes actions $A_t \in \mathcal{A}$. The environment receives actions, updates new states $S_{t+1} \in \mathcal{S}$ and outputs a scalar reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$. The agent then receives the reward, stays in new states $S_{t+1} \in \mathcal{S}$ and choses new actions $A_{t+1} \in \mathcal{A}$. This interaction continues until reaching the terminal state $S_T \in \mathcal{S}$. The goal of the agent is to derive an optimal control policy which maximizes the discounted accumulated rewards, named as expected discounted return G_t (as defined in Eq. (2)), in long term.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{2}$$

where γ is the discounting factor, $0 < \gamma < 1$.

The RL problem is based on Markov Decision Processes (MDP) [22]. An MDP has the property that the current state encodes enough information for the agent to make decision despite the previous states it has been visited. There are two main categories of RL algorithms which are policy-based ones and value-based ones. In value-based RL, a value function is used to estimate the value of being in each state. The definition of the state-value function from a given state under policy π is

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|S_t = s], \tag{3}$$

© The Institution of Engineering and Technology 2015

where $\mathbb{E}_\pi$ denotes the expectation under policy π.

Similarly, the action-value function q_π of taking action a in state s under policy π can be defined as

$$\begin{aligned} q_\pi(s,a) &\doteq \mathbb{E}_\pi[G_t|S_t=s, A_t=a] \\ &= \mathbb{E}_\pi[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}|S_t=s, A_t=a]. \end{aligned} \tag{4}$$

Among all the action-value functions, the optimal action-value function, defined as

$$q_*(s,a) \doteq \max_\pi q_\pi(s,a), \tag{5}$$

generates the optimal policy π_*.

The Q-learning Algorithm [23] is an off-policy value-based learning algorithm of RL. In this algorithm, the learned action-value function, Q, directly approximates the optimal action-value function, q^*, instead of following the current policy. The updating rule is

$$\begin{aligned} Q(S_t,A_t) &\leftarrow Q(S_t,A_t) \\ &\quad + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1},a) - Q(S_t,A_t)], \end{aligned} \tag{6}$$

where α denotes the learning rate.

The general procedures of the Q-learning algorithm is presented in Algorithm 1 [22].

Algorithm 1: Q-learning Algorithm

```
Initialize Q(s,a), ∀a ∈ A, ∀s ∈ S, arbitrarily;
For every episode;
repeat
    Initialize state S_t;
    repeat
        Perform action A_t, observe S_{t+1} and R_{t+1};
        Update Q values according to Eq. (6);
        S_t ← S_{t+1}
    until S_t reaches terminal state S_T;
until episode ends;
```

2.3 The Cart-pole Problem

The standard cart-pole problem is shown in Fig. 3. A rigid pendulum is connected to a cart with a hinge. The pendulum is able to rotate in the vertical plane while the cart is restricted to move in the one-dimension track. A horizontal force $u(t)$ is applied to this system as to balance the pendulum. There are four variables used to describe the state of the cart-pole system, which are $\theta(t)$, $\dot{\theta}(t)$, $x(t)$, $\dot{x}(t)$ denoting the angle of the pendulum, angular velocity, displacement of the cart and the velocity of the cart, respectively.

The dynamic model [24] is given below,

$$\dot{x}_1(t) = x_2(t) \tag{7}$$

$$\dot{x}_2(t) = \frac{\begin{pmatrix} -F_1(M+m)x_2(t) - m^2l^2x_2(t)^2\sin x_1(t)\cos x_1(t) \\ +F_0mlx_4(t)\cos x_1(t) + (M+m)mgl\sin x_1(t) \\ -ml\cos x_1(t)u(t) \end{pmatrix}}{(M+m)(J+ml^2) - m^2l^2(\cos x_1(t))^2} \tag{8}$$

$$\dot{x}_3(t) = x_4(t) \tag{9}$$

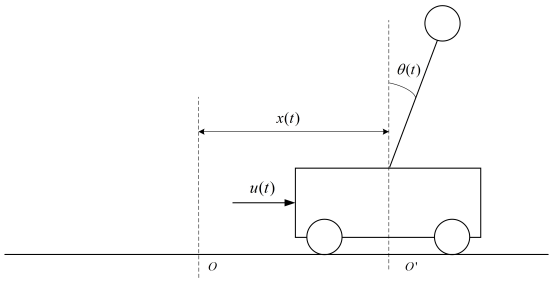

Fig. 3: Illustration of the cart-pole system

$$\dot{x}_4(t) = \frac{\begin{pmatrix} F_1mlx_2(t)\cos x_1(t) + (J+ml^2)mlx_2(t)^2\sin x_1(t) \\ -F_0((J+ml^2)x_4(t) - m^2gl^2\sin x_1(t)\cos x_1(t) \\ +(J+ml^2)u(t) \end{pmatrix}}{(M+m)(J+ml^2) - m^2l^2(\cos x_1(t))^2}, \tag{10}$$

where $x_1(t)$: the displacement of angle (rad);
$x_2(t)$: the angular velocity (rad/sec);
$x_3(t)$: the position of the cart (m);
$x_4(t)$: the linear velocity of the cart (m/s);
$u(t)$: the force applied to the cart (N);
M: the mass of the cart (kg);
m: the mass of the pendulum (kg);
l: half length of the pendulum (m);
J: the moment of inertia of the pendulum, $J = \frac{2}{3}ml$ kgm^2;
F_0: the friction factor of the cart (N/m/s);
F_1: the friction factor of the pendulum (N/rad/s);
g: the gravity acceleration, $g = 9.8$m/s^2.

3 Methodology

In this section, the details of the methodology are introduced. The whole architecture of the adaptive PID controller based on Q-learning algorithm is introduced in Section 3.1. The learning process using Q-learning algorithm is explained in Section 3.2.

3.1 Design of the Controller

The adaptive PID controller based on Q-learning algorithm proposed in this paper was designed to balance the cart-pole system. The architecture of the controller is shown in Fig. 4. The direct control of the system is accomplished by two linear PID controllers while the adaption of the parameters is based on the learned Q-tables, which are obtained from the training process using Q-learning algorithm.

As shown in Fig. 4, the control of the pendulum and the cart is implemented by PID controllers. At time step t_k, PID controller 1 takes $\theta_{ref} = 0$ as reference, inputs the error of the angle $e_p(t_k)$ defined as

$$e_p(t_k) = \theta_{ref} - \theta(t_k) \tag{11}$$

and outputs the force $u_p(t_k)$, which is defined as

$$\begin{aligned} u_p(t_k) =& K_{PP}e_p(t_k) + K_{IP}\sum_{i=0}^{k} e_p(t_i)\Delta t_k \\ &+ K_{DP}\frac{e_p(t_k) - e_p(t_{k-1})}{\Delta t_k}, \end{aligned} \tag{12}$$

where K_{PP}, K_{IP} and K_{DP} are the proportional, integral and derivative gain of PID controller 1, respectively.

PID controller 2 takes $x_{ref} = 0$ as reference, inputs the error of the position $e_c(t_k)$ defined as

$$e_c(t_k) = x_{ref} - x(t_k) \tag{13}$$

© The Institution of Engineering and Technology 2015

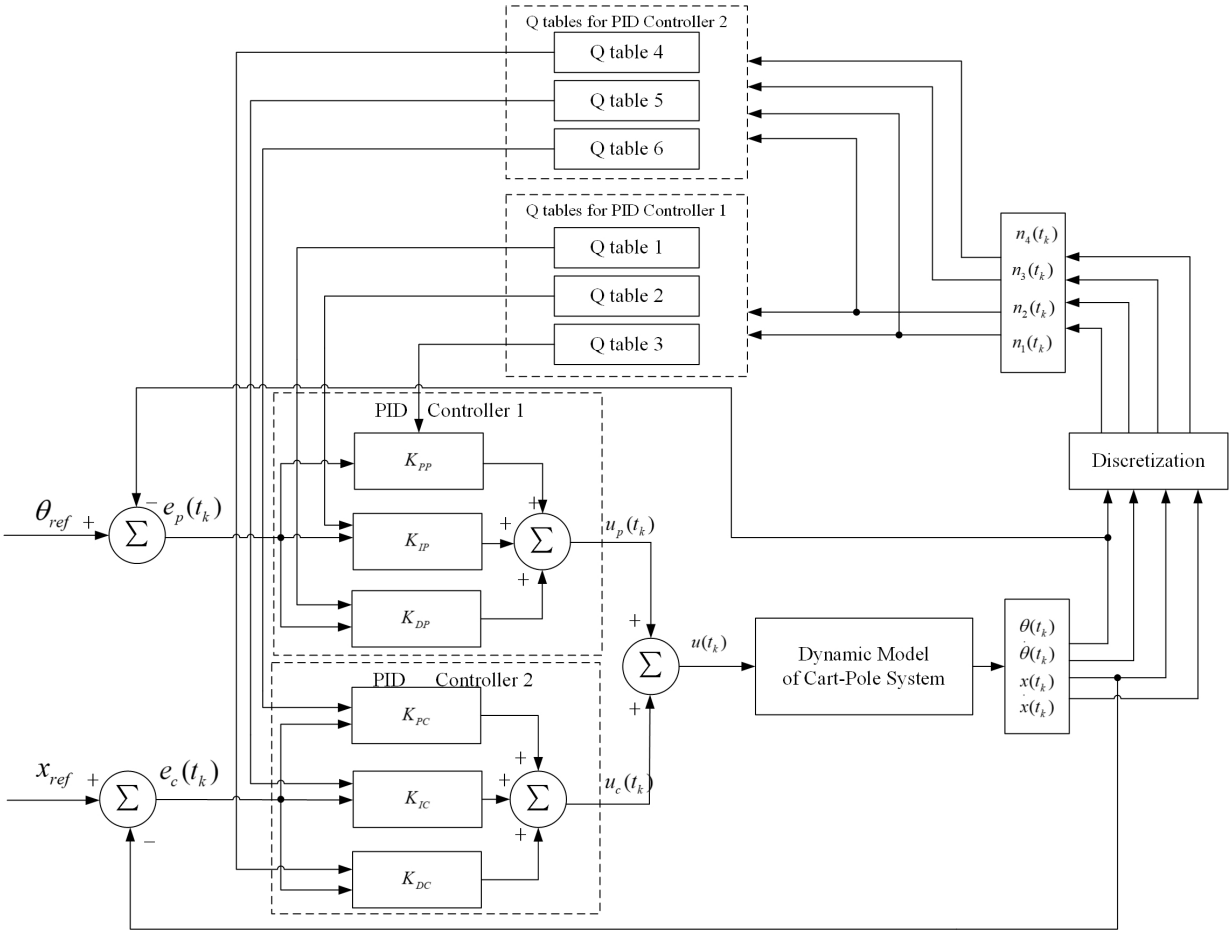

Fig. 4: The architecture of adaptive PID controller based on Q-learning algorithm

and outputs the force $u_c(t_k)$, which is defined as

$$\begin{aligned} u_c(t_k) =& K_{PC}e_c(t_k) + K_{IC}\sum_{i=0}^{k} e_c(t_i)\Delta t_k \\ &+ K_{DC}\frac{e_c(t_k) - e_c(t_{k-1})}{\Delta t_k}, \end{aligned} \tag{14}$$

where K_{PC}, K_{IC} and K_{DC} are the proportional, integral and derivative gain of PID controller 2, respectively.

The force $u_p(t_k)$ and $u_c(t_k)$ are summed up as a total force $u(t_k)$ which applies to balance the cart-pole system. By simulating the system dynamics, the states of the system $(\theta(t_k), \dot{\theta}(t_k), x(t_k), \dot{x}(t_k))$ are obtained. These four continuous variables are discretized into four discrete variables $(n_1(t_k), n_2(t_k), n_3(t_k), n_4(t_k))$, which are taken as the presentation of state $S_{t_k} \in \mathcal{S}$ in the Q-learning algorithm.

There are six learned Q tables in total with each Q table is associated with one controller gain. Q table 1 to 3 are associated to the derivative, integral and proportional gain of the PID controller 1 which controls the pendulum, respectively. Similarly, Q-table 4 to 6 are associated to the derivative, integral and proportional gain of the PID controller 2 which controls the cart, accordingly. To be noted, Q-table 1 to 3 only takes $n_1(t_k)$ and $n_2(t_k)$ as states while Q-table 4 to 6 takes the whole four variables $n_1(t_k)$, $n_2(t_k)$, $n_3(t_k)$ and $n_4(t_k)$ into consideration. When given the current state, each learned Q-table generates the optimal value for the corresponding controller gain.

3.2 *The Training Process*

One crucial procedure in designing the controller is the training of the Q tables, which maps the current states to actions (different values of gains in this situation). The whole training process is shown in pseudo code form Algorithm 2. As to speed up the learning process, an adaptive learning rate method named Delta-Bar-Delta [25] was implemented.

The following are some details of the training process.

3.2.1 Discretization: Each continuous variable is divided into several buckets, the values within the same bucket are regarded as one same state. The buckets are set using the same rule defined as

$$n = \begin{cases} 1 & \text{if } x_{con} < X_{min} \\ 10 & \text{if } x_{con} > X_{max} \\ \lfloor \frac{x_{con}}{X_{max} - X_{min}} \times N \rfloor + 1 & \text{if } X_{min} \le x_{con} \le X_{max}, \end{cases} \tag{15}$$

where $\lfloor x \rfloor = max\{n \in \mathbb{Z} | n \le x\}$; n denotes the discrete variable; x_{con} denotes the continuous variable; X_{min} and X_{max} are the lower and upper bound of x_{con}, respectively; N denotes the number of buckets each variable is divided into, $N = 10$ in this situation. The number of the buckets is decided depending on the simulation performance.

The values set for discretization are shown in Table 1.

3.2.2 $\epsilon -$ greedy Method: All six Q tables generate the actions according to $\epsilon -$ greedy method given current state. This method is defined by

$$A = \begin{cases} \text{random action} & \text{if } \xi < \epsilon \\ \arg\max_a Q(s, a) & \text{otherwise}, \end{cases} \tag{16}$$

© The Institution of Engineering and Technology 2015

Algorithm 2: The Training Process

Initialize $Q_i(s,a) = 0, \forall a \in \mathcal{A}, \forall s \in \mathcal{S}, i = 1, 2, ..., 6$;
Initialize learning rates α_1 and α_2;
Initialize exploration rate ϵ;
while $episode < maxepisode$ **do**
  $t = 0$;
  Initialize $S_t(\theta(t), \dot{\theta}(t), x(t), \dot{x}(t))$;
  Decay ϵ (when $episode > 0.6 \times maxepisode$ ϵ=0);
  for $t = 1; t \leq maxtime; t++$ **do**
    Discretization of state S_t, obtain $n_1(t), n_2(t), n_3(t), n_4(t)$;
    for $i = 1; i \leq 3; i++$ **do**
      According to $n_1(t), n_2(t)$, choose action A_i following ϵ-greedy policy;
    end
    for $i = 3; i \leq 6; i++$ **do**
      According to $n_1(t), n_2(t), n_3(t), n_4(t)$, choose action A_i following ϵ-greedy policy;
    end
    Obtain total output $u(t)$ according to Eq. 11 to 14;
    Observe new state $S_{t+1}(\theta(t+1), \dot{\theta}(t+1), x(t+1), \dot{x}(t+1))$;
    Receive reward R_p for $Q_1(s,a), Q_2(s,a)$ *and* $Q_3(s,a)$;
    Receive reward R_c for $Q_4(s,a), Q_5(s,a)$ *and* $Q_6(s,a)$;
    Discretization of state S_{t+1}, obtain $n_1(t+1), n_2(t+1), n_3(t+1), n_4(t+1)$;
    Update learning rate α_1 for $Q_1(s,a), Q_2(s,a)$ *and* $Q_3(s,a)$;
    Update learning rate α_2 for $Q_4(s,a), Q_5(s,a)$ *and* $Q_6(s,a)$;
    Update $Q_1(s,a), Q_2(s,a)$ *and* $Q_3(s,a)$ using R_p and α_1;
    Update $Q_4(s,a), Q_5(s,a)$ *and* $Q_6(s,a)$ using R_c and α_2;
    $S_t \leftarrow S_{t+1}$
  end
end

Table 1 Values set for discretization

Values	X_{min}	X_{max}
$\theta(t)$	$-45 \times \pi/180$	$45 \times \pi/180$
$\dot{\theta}(t)$	-15	15
$x(t)$	-3	3
$\dot{x}(t)$	-15	15

where ξ is a random number subject to normal distribution, $0 \leq \xi \leq 1$.

As to speed up the convergence, the value of ϵ decays with the increment of training episodes and is set to zero after certain episodes while the number of episodes is decided according to the training performance. The details are defined as

$$\epsilon(eps) = \begin{cases} \frac{1}{1+e^{eps}} + 0.001 & eps < 0.6 \times maxepisode \\ 0 & \text{otherwise,} \end{cases} \tag{17}$$

where eps is the current episode; $maxepisode$ is the maximum number of episodes.

3.2.3 Reward Scheme: The reward schemes are different according to the PID controllers. The Q tables for pendulum share the same reward scheme while the others apply a different one. The reward scheme for the Q tables that are associated with the gains of PID controller 1 is defined as

$$R_p = \begin{cases} 1 & \text{if } |\theta(t+1)| < |\theta(t)| \text{ and } |\theta(t+1)| > \theta_{lim} \\ 2 & \text{if } |\theta(t+1)| \leq \theta_{lim} \\ 0 & \text{otherwise,} \end{cases} \tag{18}$$

where θ_{lim} is the threshold set for angle, $\theta_{lim} = 0.01$ in this situation.

The reward scheme for the Q tables that are associated with the gains of PID controller 2 is defined as

$$R_c = \begin{cases} 1 \text{ if } |\theta(t+1)| < |\theta(t)| \text{ and } |x(t+1)| < |x(t)| \text{ and } |x(t+1)| > x_{lim} \\ 2 \text{ if } |x(t+1)| \leq x_{lim} \\ 0 \text{ otherwise,} \end{cases} \tag{19}$$

where x_{lim} is the threshold set for position, $x_{lim} = 0.1$ in this situation.

Table 2 Parameters set for adaptive learning rate algorithm

	Q tables of PID controller 1	Q tables of PID controller 2
α_0	0.015	0.3
κ	$0.1\alpha_0$	$0.002\alpha_0$
ϕ	0.5	0.5
φ	0.5	0.5

3.2.4 Adaptive Learning Rate: As to improve the efficiency of convergence, the adaptive learning rate algorithm called Delta-Bar-Delta [25] was implemented. The algorithm is defines as

$$\Delta\alpha_t = \begin{cases} \kappa, & \text{if } \bar{\delta}_{t-1}\delta_t > 0 \\ -\phi\alpha_t, & \text{if } \bar{\delta}_{t-1}\delta_t < 0 \\ 0, & \text{if } \bar{\delta}_{t-1}\delta_t = 0 \end{cases} \tag{20}$$

where $\Delta\alpha_t$ is the increment of the learning rate in time step t;
κ is a positive constant value to increase the learning rate;
ϕ is a positive constant value denoting the discounting factor;
δ_t is the temporal difference (TD) error [22] in time step t, $\delta_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)$;
$\bar{\delta}_t = (1-\varphi)\delta_t + \varphi\bar{\delta}_{t-1}$.

The learning rate in time step $t+1$ is updated as

$$\alpha_{t+1} = \alpha_t + \Delta\alpha_t. \tag{21}$$

By applying this rule, the learning rate will be updated by comparing the current TD error with the accumulated TD error in previous steps. When the learning rate becomes too large, the increment of the learning rate changes sign and reduces the learning rate. On the other hand, if the learning rate is too small, the learning rate keeps changing in the previous trend and speeds up the convergence.

All Q tables use adaptive learning rate algorithm while the setting of parameters are different. Q tables used to change the gains of the controller of the pendulum share the same set of parameters while the other Q tables share another set of parameters. The values used in this research are presented in Table 2.

3.2.5 Other Settings of Parameters: The parameters used in initialization are listed in Table 3. The running of each episode will be interrupted if one of the following situations occur, $|\theta(t)| > 0.5\pi$ or $|x(t)| > 10.0$, which are not ideal in real physical world as to improve the efficiency of the simulation.

4 Simulation Results

In this section, the simulation results of implementing adaptive PID controller based on Q-learning algorithm (named as QPID controller in the following content for the simplicity of expression) are demonstrated. The QPID controller was trained from

© The Institution of Engineering and Technology 2015

Table 3 Parameters set in initialization

Variables	Values
Initial position of $\theta(t)$	$-0.7854, +0.7854$
Initial position of $\dot{\theta}(t)$	0
Initial position of $x(t)$	$-0.1, +0.1$
Initial position of $\dot{x}(t)$	0
Number of actions for all tables	50
Variation range of K_{PP}	$[-3000, 0]$
Variation range of K_{IP}	$[-300, 0]$
Variation range of K_{DP}	$[-3000, 0]$
Variation range of K_{PC}	$[-2000, 0]$
Variation range of K_{IC}	$[-5, 0]$
Variation range of K_{DC}	$[0, 2000]$
Maximum episodes	6000
Maximum timesteps	8000
Interval sampling time	10ms
γ	0.99

Table 4 Parameters of cart-pole system

Variables	Values
M	$1.3282kg$
m	$0.22kg$
l	$0.304m$
F_0	$22.915N/m/s$
F_1	$0.007056N/rad/s$
g	$9.8m/s^2$

four fixed initial positions ($\theta(t) = \pm 0.7854rad, \dot{\theta}(t) = 0, x(t) = \pm 0.05m, \dot{x}(t) = 0$) with $0 - 0.04rad$ subtracted from each fixed point randomly on angle as disturbances. The learned Q tables were implemented to generate the optimal gains of the PID controllers. The QPID controller was applied to balance the cart-pole system from a series of initial positions, which are the combinations of $\theta(t) \in \{\pm 0.7854, \pm 0.5236, \pm 0.2618, 0rad\}, \dot{\theta}(t) = 0, x(t) \in \{\pm 0.05, \pm 0.02, 0m\}, \dot{x}(t) = 0$, and succeded in balancing the system from all these initial positions. Additionally, as for comparison, two other types of controllers are applied to the system to compare the controlling performances. The first type of controller employs the tabular Q-learning algorithm (named as Q controller in the following content for the simplicity of expression) and the second one is the conventional PID controller. Initial position $\theta(t) = 0.1745rad, \dot{\theta}(t) = 0, x(t) = 0, \dot{x}(t) = 0$ was chosen as the initial position to compare the performance of QPID controller with Q controller, considering that the Q controller has the limitation and is not able to balance the system when the angle or the position exceeds the set value. Initial position $x_1 = 45deg, x_2 = 0, x_3 = 0.05m, x_4 = 0$ was chosen as the initial position to test the performance of QPID controller against the PID controller. The structure of the following content is as follows. Section 4.1 provides the setting of the parameters in the simulation, which includes parameters set for the cart-pole system and the controllers used in comparison. Section 4.2 compares the performance of QPID controller against Q controller and the performance of the QPID controller against PID controller, separately.

4.1 Setting of Parameters

4.1.1 Cart-pole System: The parameters set for the cart-pole system are illustrated in Table 4.

4.1.2 Q Controller: The Q controller takes the four variables of the cart-pole system $(\theta(t), \dot{\theta}(t), x(t), \dot{x}(t))$ as input and generates the force which directly applies to the cart to balance the system. $\theta(t)$ is divided into 6 buckets (with $-21 \times pi/180, -6 \times pi/180, 0, 6 \times pi/180, 21 \times pi/180$ as dividing points); $\dot{\theta}(t)$ is divided into 3 buckets (with $\pm 6 \times pi/180$ as dividing points); $x(t)$ is divided into 3 buckets (with ± 2.4 as dividing points); $\dot{x}(t)$ is divided into 3 buckets (with ± 0.5 as dividing points). The output force is mapped into 10 actions which evenly distribute among $[-100N, +100N]$. The system receives reward as 1 when $|\theta(t)| < 6 \times pi/180$ or $|x(t)| < 0.1$, otherwise, reward is 0.

The relevant parameters set for Q controller is given in Table 5.

Table 5 Parameters of Q controller

Variables	Values
Initial position of $\theta(t)$	0.1745
Initial position of $\dot{\theta}(t)$	0
Initial position of $x(t)$	0
Initial position of $\dot{x}(t)$	0
Maximum episodes	30000
Maximum time step	2000
Interval sampling time	$10ms$
Discount factor γ	0.99
Learning rate α	0.01
Exploration rate ϵ	$min(1, 500/episode)$

Table 6 Parameters of PID controllers

Gains	PID controller 1	PID controller 2
P	-270	-80
I	-10	-0.5
D	-1500	2000

4.1.3 PID Controller: The PID scheme applied in this research consists of two linear PID controllers, which control the angle position of the pole and the position of the cart separately. The gains of two linear PID controllers were obtained by manually tuned according to Ziegler-Nichols method [9] and are shown in Table 6.

4.2 Controlling Performance

4.2.1 Learning Curves: The learning performance of Q-learning algorithm is indicated by the learning curve, which plots the accumulated reward obtained with the incrementation of the episode. The aim of comparing learning curves is to indicate the learning efficiency of Q-learning algorithm applied in different controllers. The convergence of the learning curve shows the success in Q-learning algorithm and the time steps used to reach convergence indicates the learning efficiency. The learning curve of QPID controller in the training process is presented in Fig. 5 and the learning curve of the Q controller is shown in Fig. 6, both of which show the convergence. However, it can be observed that the learning curve of the QPID controller does not show the convergence trend as the conventional learning curves does. This is due to the actions chose by the Q tables are the gains of PID controller instead of the forces. There are several combinations of gains which would be suitable for the PID controller to balance the system and reduce the difficulty of the Q-learning algorithm in finding proper set of gains for the PID controllers, which results in the high frequency of appearance of high accumulated reward in the learning curve. Nevertheless, after about $5,000$ time steps, the exploration rate of the Q-learning algorithm used in QPID controller was set to zero as to improve the efficiency of converge, an obvious convergence can be noticed, which indicates the success of Q-learning algorithm in finding optimal solutions to adapting the gains.

4.2.2 Performances of Controllers: The learned Q tables were implemented in the designed controllers to balance the system. The following shows the comparison of different controlling performances by plotting the response curves of cart-pole system applied with different controllers. In each comparison, four response curves are given by plotting the position and velocity of both the pole and the cart against time in one figure, which indicates the characteristics of the controllers.

(1) Comparison of Q controller with QPID controller

Fig. 7 compares the response curve of the system controlled by Q controller and QPID controller with the initial position $\theta(t) =$

© The Institution of Engineering and Technology 2015

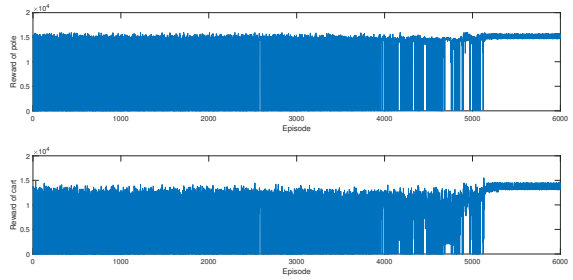

Fig. 5: Learning curve of QPID controller

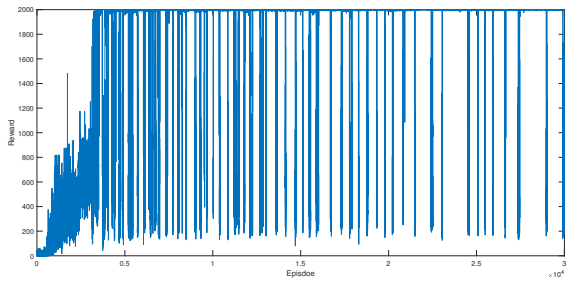

Fig. 6: Learning curve of Q Controller

Table 7 Characteristics of response curves (Q controller and QPID controller)

Characteristics	Q controller	QPID controller
Overshoot (angle)	$0.0479(rad)$	$0.3763(rad)$
Undershoot (angle)	$0.1316(rad)$	$0.1574(rad)$
Rise time (angle)	$6(time\ step)$	$4(time\ step)$
Settling time (angle)	$4(time\ step)$	$69(time\ step)$
Steady error(angle)	$0.026(rad)$	$0(rad)$
Overshoot (position)	$0.0930(m)$	$0.2121(m)$
Undershoot (position)	$0.2443(m)$	$0.4264(m)$
Rise time (position)	$76(time\ step)$	$86(time\ step)$
Settling time (position)	$-$	$772(time\ step)$
Steady error (position)	$0.1351(m)$	$0(m)$

$0.1745rad, \dot{\theta}(t) = 0, x(t) = 0, \dot{x}(t) = 0$. The characteristics of the response curves are shown in Table 7. The overshooting or undershooting of the curves related to the QPID controller are greater than the ones related to the Q controller both in pole and cart position. Besides, the settling time of the QPID controller (69 $timestep$ for pole and 86 $timestep$ for cart) is longer than the Q controller (4 $timestep$ for pole and 76 $timestep$ for cart). One factor that leads to these performances is the initial position in this test was not trained in the learning process, the combination of the gains might not be the optimal solution to the controllers from this starting position. However, QPID shows an obvious advantage in steady state since Q PID controller reaches the desired position both for the pole and the cart while Q controller remains an error of $0.026rad$ for pole and $0.1351m$ for cart with obvious oscillation. In conclusion, the Q PID controller has bigger overshooting and undershooting compared with Q controller and longer settling time but has better performance in reaching steady state and reducing steady-state error.

The total forces that generated by two controllers are shown in Fig. 8.

The adaption of the gains of QPID controller during controlling process is presented in Fig. 9, where the red crosses denote the chosen values of the gains of PID controllers in each time step t and the blue lines indicate the trend of variation.

(2) Comparison of PID controller with QPID controller

Table 8 Characteristics of response curves (PID controller and QPID controller)

Characteristics	PID	QPID
Overshoot (angle)	$0.3527(rad)$	$0.5866(rad)$
Undershoot (angle)	$0.1316(rad)$	$0.4306(rad)$
Rise time (angle)	$44(time\ step)$	$4(time\ step)$
Settling time (angle)	$320(time\ step)$	$64(time\ step)$
Steady error (angle)	$0(rad)$	$0(rad)$
Overshoot (position)	$2.075(m)$	$0.9279(m)$
Undershoot (position)	$0.2443(m)$	$0.1807(m)$
Rise time (position)	$76(time\ step)$	$13(time\ step)$
Settling time (position)	$426(time\ step)$	$772(time\ step)$
Steady error (position)	$0(m)$	$0(m)$

The initial position set in this comparison is $\theta(t) = 0.7854rad, \dot{\theta}(t) = 0, x(t) = 0.1m, \dot{x}(t) = 0$. Fig. 10 shows the response curves of the cart-pole system when QPID controller and PID controller were applied to control. The characteristics of the response curves are illustrated in Table 8. The QPID controller results in bigger overshoot in balancing the pole while smaller overshoot and undershoot in balancing the position of the cart. The settling time in both QPID controller and PID controller are almost the same.

The output forces generated by QPID controller and PID controller are shown in Fig. 11.

Fig. 12 demonstrates the adaption of the gains of QPID controller during controlling process.

5 Conclusion

In this research, an adaptive PID controller based on Q-learning algorithm was proposed. The performance of the controller was tested on the cart-pole problem with comparison of conventional PID controller and Q controller. According to the simulation results, QPID controller designed in this research was trained in four fixed positions and was able to balance the system starting from various different initial positions, which shows the advantage in generality of adapting the changes made to systems. Two fixed initial positions were chosen to compare controlling performance with the conventional PID controller and the Q controller, where the QPID controller shows relatively equivalent performance or even better in stabilization of achieving smaller steady-state error. However, there are tiny oscillations observed and obvious overshooting and undershooting in the controlling performance using QPID controller, which could lead to unstable behavior of the system and need to be improved in the future research. Besides, the learning efficiency is considered to be improved. Additionally, future research plans to apply this QPID controller to various different applications such as the mass-spring-damper system, beam and ball system, multiple-link arms, etc., to improve the generality.

6 Acknowledgement

This work was partly supported by King's College London, China Scholarship Council and the Ministry of Science and Technology, Taiwan (Grant MOST-106-2221-E-027-009).

© The Institution of Engineering and Technology 2015

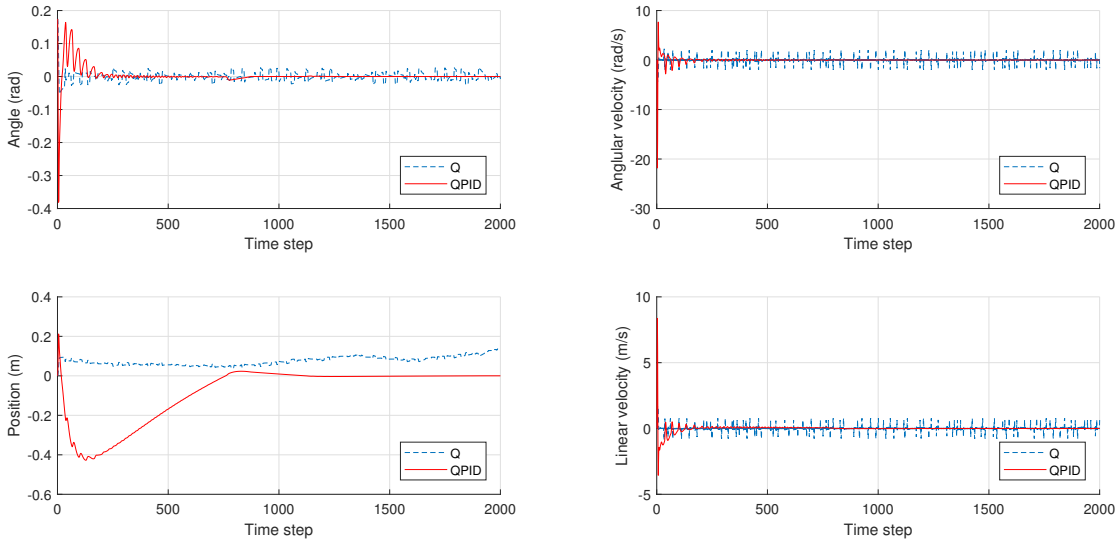

Fig. 7: Response curves of Q controller and QPID controller

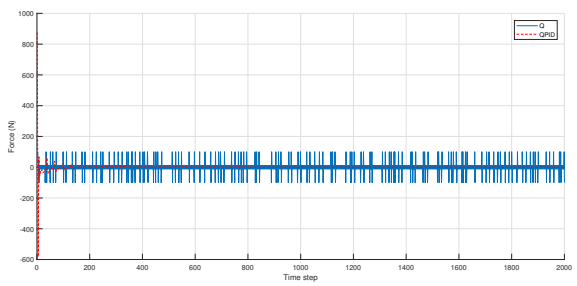

Fig. 8: Forces generated by Q controller and QPID controller

© The Institution of Engineering and Technology 2015

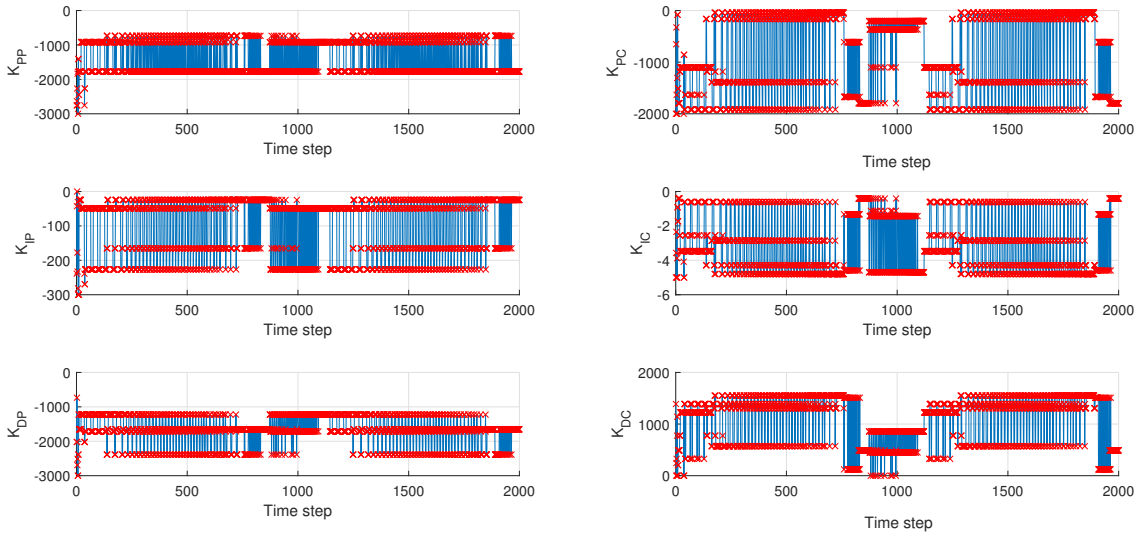

Fig. 9: Adaption of the gains of QPID controllers (initial position $\theta(t) = 0.1745rad, \dot{\theta}(t) = 0, x(t) = 0, \dot{x}(t) = 0$)

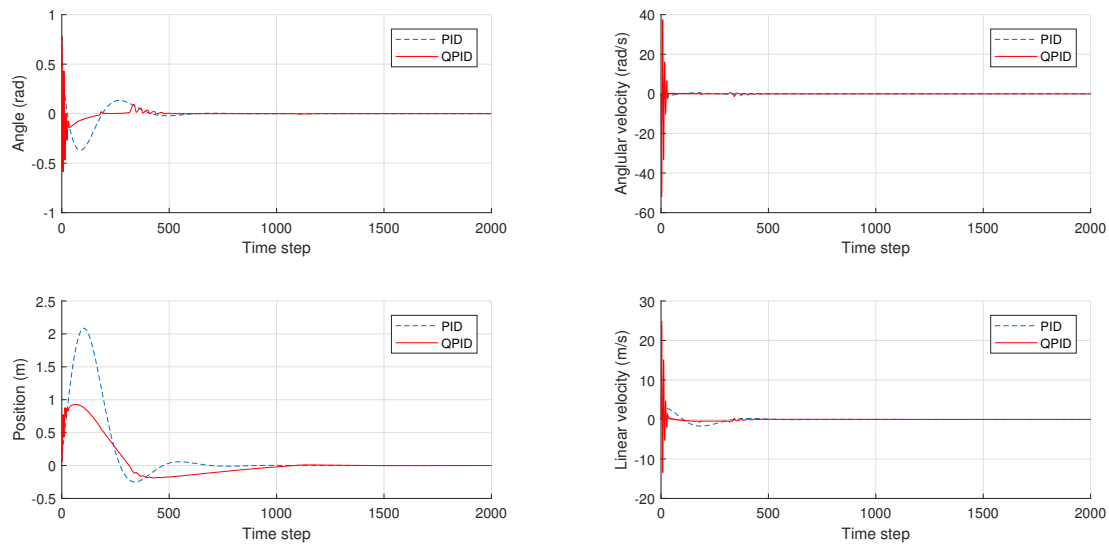

Fig. 10: Response curves of PID controller and QPID controller

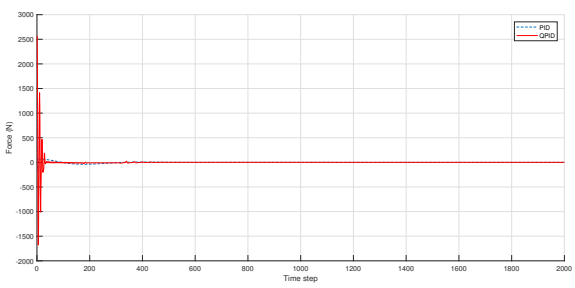

Fig. 11: Forces generated by PID controller and QPID controller

© The Institution of Engineering and Technology 2015

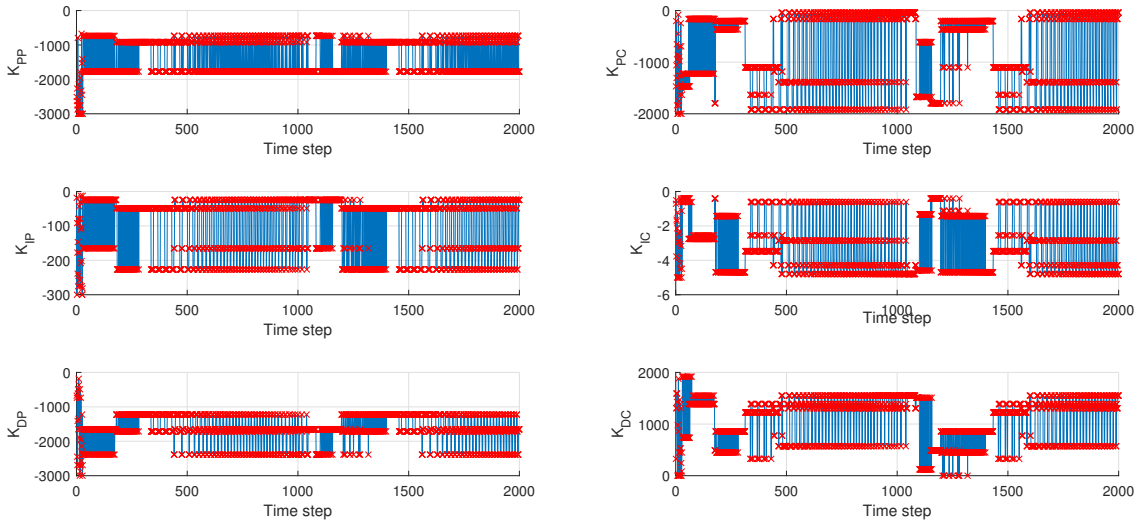

Fig. 12: Adaption of the gains of QPID controllers (initial position $\theta(t) = 0.7854rad, \dot{\theta}(t) = 0, x(t) = 0.1m, \dot{x}(t) = 0$)

© The Institution of Engineering and Technology 2015

7 References

1 M. F. Miranda and K. G. Vamvoudakis, "Online optimal auto-tuning of PID controllers for tracking in a special class of linear systems," in *American Control Conference (ACC), 2016*, pp. 5443–5448, IEEE, 2016.

2 I. Carlucho, M. De Paula, S. A. Villar, and G. G. Acosta, "Incremental Q-learning strategy for adaptive PID control of mobile robots," *Expert Systems with Applications*, vol. 80, pp. 183–199, 2017.

3 M. Shadi and M. Sargolzaei, "Application of reinforcement learning to improve control performance of plant," in *2008 IEEE International Conference on Computational Intelligence for Measurement Systems and Applications*.

4 T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

5 J. A. Bagnell and J. G. Schneider, "Autonomous helicopter control using reinforcement learning policy search methods," in *Proc. of the IEEE International Conference on International Conference on Robotics and Automation*, vol. 2, pp. 1615–1620, IEEE, 2001.

6 B. Fernandez-Gauna, I. Ansoategui, I. Etxeberria-Agiriano, and M. Graña, "Reinforcement learning of ball screw feed drive controllers," *Engineering Applications of Artificial Intelligence*, vol. 30, pp. 107–117, 2014.

7 P. Ramanathan, K. K. Mangla, and S. Satpathy, "Smart controller for conical tank system using reinforcement learning algorithm," *Measurement*, vol. 116, pp. 422–428, 2018.

8 H. Boubertakh, M. Tadjine, P.-Y. Glorennec, and S. Labiod, "Tuning fuzzy pd and pi controllers using reinforcement learning," *ISA Transactions*, vol. 49, no. 4, pp. 543–551, 2010.

9 K. J. Åström and T. Hägglund, *PID controllers: Theory, Design, and Tuning*, vol. 2. Instrument society of America Research Triangle Park, NC, 1995.

10 C. J. Wu, T. L. Lee, Y. Y. Fu, and L. C. Lai, "Auto-tuning fuzzy PID control of a pendubot system," in *2007 IEEE International Conference on Mechatronics*, pp. 1–6, May 2007.

11 H. B. Duan, D. B. Wang, and X. F. Yu, "Novel approach to nonlinear pid parameter optimization using ant colony optimization algorithm," *Journal of Bionic Engineering*, vol. 3, no. 2, pp. 73–78, 2006.

12 H. A. Varol and Z. Bingul, "A new PID tuning technique using ant algorithm," in *Proceedings of the 2004 American Control Conference*, vol. 3, pp. 2154–2159 vol.3, June 2004.

13 M. N. Howell, G. P. Frost, T. J. Gordon, and Q. H. Wu, "Continuous action reinforcement learning applied to vehicle suspension control," *Mechatronics*, vol. 7, no. 3, pp. 263–276, 1997.

14 S. M. A. Mohammadi, A. A. Gharaveisi, M. Mashinchi, and S. M. R. Rafiei, "New evolutionary methods for optimal design of PID controllers for AVR system," in *2009 IEEE Bucharest PowerTech*, pp. 1–8, June 2009.

15 F. M. Pour and A. A. Gharaveisi, "Opposition-based discrete action reinforcement learning automata algorithm case study: optimal design of a PID controller," *Turkish Journal of Electrical Engineering & Computer Sciences*, vol. 21, no. 6, pp. 1603–1614, 2013.

16 M. N. Howell and M. C. Best, "On-line PID tuning for engine idle-speed control using continuous action reinforcement learning automata," *Control Engineering Practice*, vol. 8, no. 2, pp. 147–154, 2000.

17 S. Mohammadi, A. Gharaveisi, M. Mashinchi, and V.-N. SM, "Development of a novel reinforcement learning automata method for optimum design of proportional integral derivative controller for nonlinear systems," in *Proceedings of the World Congress on Engineering*, vol. 3, 2008.

18 M. Sedighizadeh and A. Rezazadeh, "Adaptive PID controller based on reinforcement learning for wind turbine control," in *Proceedings of World Academy of Science, Engineering and Technology*, vol. 27, pp. 257–262, 2008.

19 C. J. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

20 M. Esmaeili, H. Shayeghi, H. Mohammad Nejad, and A. Younesi, "Reinforcement learning based PID controller design for LFC in a microgrid," *COMPEL-The international Journal for Computation and Mathematics in Electrical and Electronic Engineering*, vol. 36, no. 4, pp. 1287–1297, 2017.

21 A. el Hakim, H. Hindersah, and E. Rijanto, "Application of reinforcement learning on self-tuning PID controller for soccer robot multi-agent system," in *2013 Joint International Conference on Rural Information Communication Technology and Electric-Vehicle Technology (rICT ICeV-T)*, pp. 1–6, Nov 2013.

22 R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge, 1998.

23 C. J. C. H. Watkins, *Learning from delayed rewards*. PhD thesis, King's College, Cambridge, 1989.

24 H. K. Lam and H. F. H. Leung, "Fuzzy controller with stability and performance rules for nonlinear systems," *Fuzzy Sets and Systems*, vol. 158, no. 2, pp. 147–163, 2007.

25 R. A. Jacobs, "Increased rates of convergence through learning rate adaptation," *Neural networks*, vol. 1, no. 4, pp. 295–307, 1988.

© The Institution of Engineering and Technology 2015